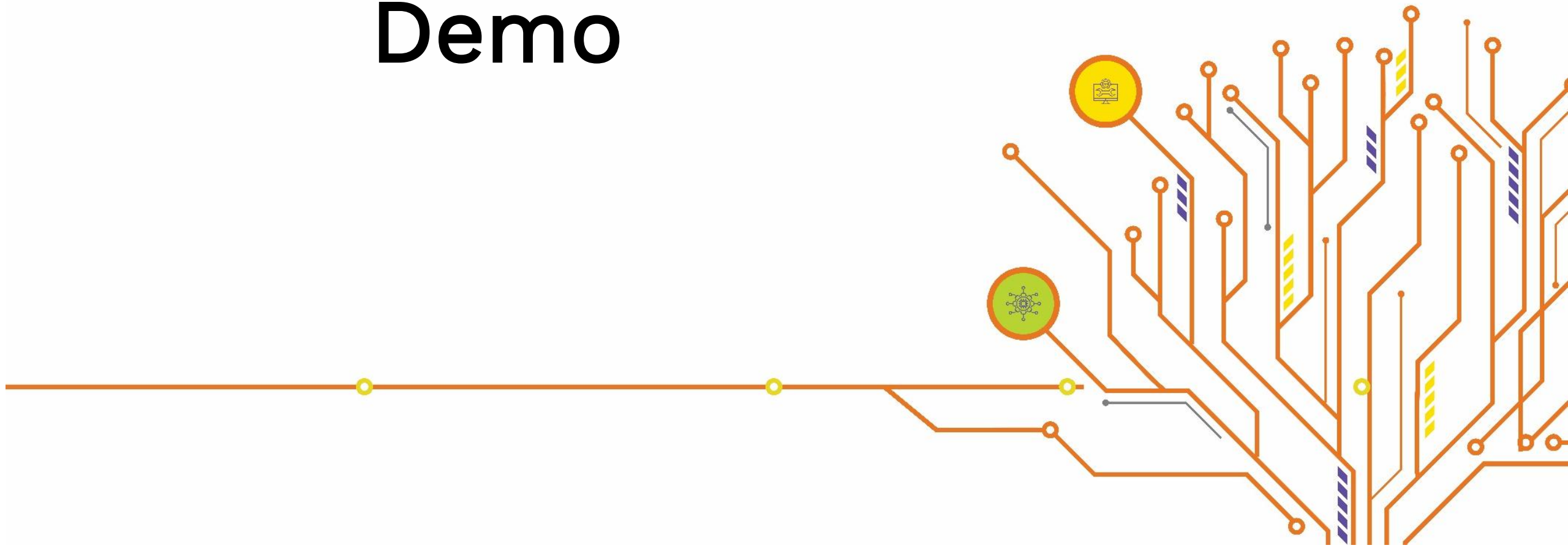
KPIT | APEX LAB

Understanding Demo



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

User Programs Access Devices: Syscalls (1/3)

Trace how a user program interacts with a device via syscalls → VFS → driver.

1. Trace syscalls with strace

```
$ cd ~
$ mkdir demo && cd demo
$ strace -o trace.txt -f cat /dev/zero | head -c 1024 > /tmp/junk
$ sed -n '1,60p' trace.txt
```

What this does:

- cat /dev/zero → produces an infinite stream of zero bytes.
- head -c 1024 → consumes only the first 1024 bytes, then exits.
- strace -o trace.txt -f → records syscalls from cat (and its children if any; head isn't a child here, so it won't be traced).
- Output redirected to /tmp/junk.

User Programs Access Devices: Syscalls (2/3)

Terminal will show details like this:

407636 execve("/usr/bin/cat", ["cat", "/dev/zero"], ...) = 0 → User-space program starts.

407636 openat(AT_FDCWD, "/dev/zero", O_RDONLY) = 3 → User space requests open(); kernel VFS routes to /dev/zero driver.

407636 read(3, "\177ELF\2\1\1\3...", 832) = 832 407636 read(3, "\0\0\0\0\0\0\0...", 131072) = 131072 → User space calls read(); kernel returns bytes. For /dev/zero, it *generates zeros*.

407636 write(1, "\0\0\0\0\0\0\0...", 131072) = 65536 → User space calls write(); kernel writes to a pipe (stdout → head).

407636 --- SIGPIPE {si_signo=SIGPIPE, ...} ---

407636 +++ killed by SIGPIPE +++ → head consumed 1024 bytes and closed the pipe; kernel signals cat with SIGPIPE.

Key mapping:

- User space: cat issues open, read, write calls.
- Kernel space: VFS + device driver /dev/zero + pipe implementation handle the actual work.
- write() returning 65536 (not 131072) is due to pipe capacity.

cat --read --> /dev/zero driver
--write --> pipe buffer
head reads 1024 bytes → closes → SIGPIPE to cat

User Programs Access Devices: Syscalls (3/3)

2. Execute

```
grep '/dev/zero' trace.txt        # Show open()
grep 'read(' trace.txt | head     # Show read() calls
grep 'write(' trace.txt | head    # Show write() calls
```

Observation:

```
423993 execve("/usr/bin/cat", ["cat", "/dev/zero"], 0x7ffece84b420 /* 48 vars */) = 0
423993 openat(AT_FDCWD, "/dev/zero", O_RDONLY) = 3
423993 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0\0"..., 832) = 832 423993 read(3,
"\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 131072) = 131072 423993 write(1,
"\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 131072) = 65536
```

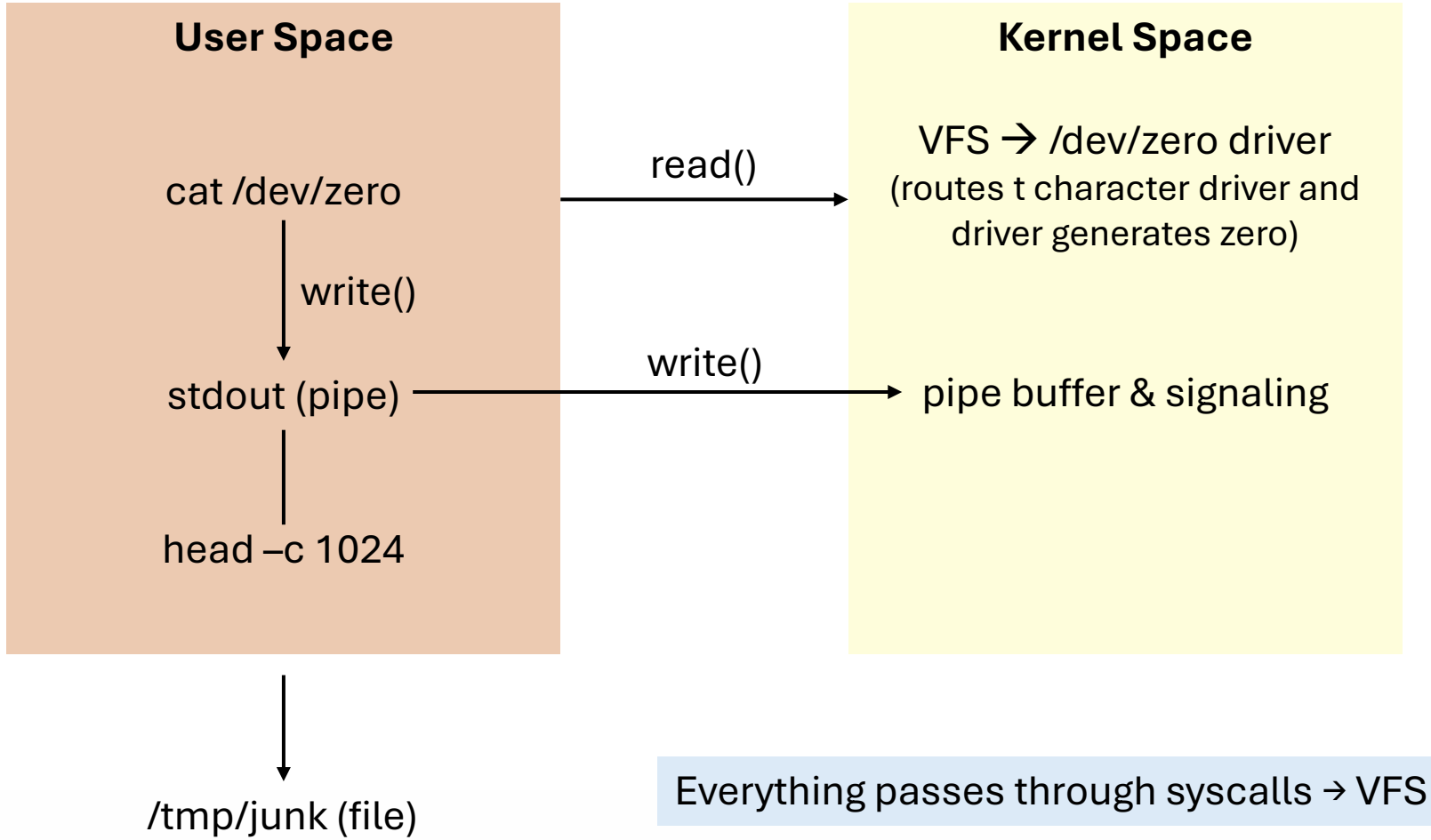
read(3, buf, 131072) = 131072

- 3 → file descriptor (FD) for /dev/zero.
- 131072 (inside) → requested size.
- = 131072 (after) → actual bytes returned by kernel.

write(1, buf, 131072) = 65536

- 1 → FD for stdout (a **pipe** in this pipeline).
- Requested 131072 bytes, kernel actually wrote 65536 (pipe capacity/consumer behavior).

User → Kernel → Device



Everything passes through syscalls → VFS → driver.

Why /dev/zero ≠ Regular File?

Why does /dev/zero behave differently from a regular file?

- /dev/zero is a **character device** exposed via a **special device node**.
- /dev/zero is part of major number 1 → memory devices
- It's backed by a **kernel driver** that returns zero bytes on read **indefinitely**.
- No disk access occurs; it's generated in kernel space.

A regular file has finite size stored on disk; /dev/zero is infinite and virtual.

Kernel Internals via Virtual Filesystems (1/)

/proc & /sys

```
$ uname -a
Linux PL-9L 6.8.0-90-generic #91~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Nov 20 15:20:45 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
```

- Linux → Kernel name.
- PL-9L → Hostname of your machine.
- 6.8.0-90-generic → Kernel version.
- #91~22.04.1-Ubuntu → Build number and Ubuntu-specific patch level.
- SMP → Symmetric Multi-Processing (supports multiple CPUs).
- PREEMPT_DYNAMIC → Kernel preemption model (dynamic preemption).
- Thu Nov 20 15:20:45 UTC 2 → Kernel build date/time.
- x86_64 x86_64 x86_64 → Architecture: kernel, CPU, and platform are all 64-bit.
- GNU/Linux → Operating system type.

Kernel Internals via Virtual Filesystems

```
$ cat /proc/meminfo
```

- MemTotal: 32605452 kB → Total physical RAM (~32 GB).
- MemFree: 17589048 kB → Free memory not used by anything (~17 GB).
- MemAvailable: 22411248 kB → Memory available for new apps (~22 GB).
- Buffers: 180696 kB → Memory used for block device buffers.
- Cached: 6041868 kB → Memory used for page cache (file data).
- SwapCached: 68 kB → Swap pages cached in RAM.
- Active: 10223932 kB → Memory actively used by processes.
- Inactive: 3445064 kB → Memory not actively used but still allocated.
- Active(anon): 8538800 kB → Anonymous memory actively used (heap, stack).
- Inactive(anon): 75312 kB → Anonymous memory inactive.

Key point: These values are **generated by the kernel on demand**, not stored on disk.

Kernel Internals via Virtual Filesystems

```
cat /proc/meminfo
```

- MemTotal: 32605452 kB → Total physical RAM (~32 GB).
- MemFree: 17589048 kB → Free memory not used by anything (~17 GB).
- MemAvailable: 22411248 kB → Memory available for new apps (~22 GB).
- Buffers: 180696 kB → Memory used for block device buffers.
- Cached: 6041868 kB → Memory used for page cache (file data).
- SwapCached: 68 kB → Swap pages cached in RAM.
- Active: 10223932 kB → Memory actively used by processes.
- Inactive: 3445064 kB → Memory not actively used but still allocated.
- Active(anon): 8538800 kB → Anonymous memory actively used (heap, stack).
- Inactive(anon): 75312 kB → Anonymous memory inactive.

Key point: These values are **generated by the kernel on demand**, not stored on disk.

Kernel Internals via Virtual Filesystems

```
$ cat ls /sys/class
```

accel, block, net, power_supply, thermal, tty, usbmisc …

Explanation:

- Each directory represents a **device class** exposed by sysfs.
- Examples:
    - block → Block devices (disks).
    - net → Network interfaces.
    - thermal → Thermal sensors.
    - tty → Terminals.
    - power_supply → Battery/AC power info.

Kernel Internals via Virtual Filesystems

```
ls /sys/class/net
```

- enp0s31f6 lo virbr0 wlp0s20f3 zcctun0

Explanation:

- enp0s31f6 → Wired Ethernet interface.
- lo → Loopback interface (127.0.0.1).
- virbr0 → Virtual bridge (used by virtualization).
- wlp0s20f3 → Wireless interface.
- zcctun0 → Likely a VPN or tunnel interface.

Is /proc/meminfo stored on disk?

- No, it’s generated by the kernel on demand.

Device Nodes & Major/Minor Numbers

```
ls -l /dev/null /dev/zero /dev/tty
```

```
crw-rw-rw- 1 root root 1, 3 Jan 15 11:20 /dev/null
crw-rw-rw- 1 root tty  5, 0 Jan 15 11:20 /dev/tty
crw-rw-rw- 1 root root 1, 5 Jan 15 11:20 /dev/zero
```

- **Leading c** → Character device (stream-oriented, e.g., /dev/null, /dev/zero, /dev/tty).
- If it were b, it would be a block device (disk-like, e.g., /dev/sda).
- **Permissions (rw-rw-rw-)** → Anyone can read/write these devices.
- **Owner/Group (root root)** → Owned by root.
- **1, 3 or 1, 5** → **Major, Minor numbers**:
- **Major** = Identifies the driver in the kernel (e.g., 1 = memory devices).
- **Minor** = Identifies the specific device handled by that driver (3 = /dev/null, 5 = /dev/zero).
- **Path** → /dev/null, /dev/zero, /dev/tty are **device nodes**, not regular files.

What happens if you delete /dev/null?

- You lose the handle to the driver;
- programs that rely on /dev/null (for discarding output) will fail.
- The driver still exists in the kernel, but the user-space entry point is gone.

Show major/minor mapping

```
cat /proc/devices | head
```

- This lists all registered major numbers and their drivers.

annotated explanation for each entry in your /proc/devices snippet:

1 mem
→ Major number 1 is assigned to yesthe "memory" driver family.
Devices under this include:
- /dev/null (discard data)
- /dev/zero (generate zeros)
- /dev/full, /dev/random, /dev/urandom
These are all character devices that interact with kernel memory.

4 /dev/vc/0
→ Major number 4 is for virtual consoles.
/dev/vc/0 is the first virtual console device (used for text terminals).

4 tty
→ Also under major 4: TTY devices (teletype terminals).
These represent physical or virtual terminal interfaces.

4 ttyS
→ Serial TTY devices (e.g., /dev/ttyS0 for COM ports).
Used for serial communication.

5 /dev/tty
→ Major number 5 is for system TTY devices.
/dev/tty refers to the controlling terminal for the current process.

5 /dev/console
→ The system console device (kernel messages and login prompt).
Critical for system interaction during boot.

5 /dev/ptmx
→ Pseudo-terminal master multiplexer.
Used to create new pseudo-terminal pairs for terminal emulators.

5 ttyprintk
→ Special TTY device that prints kernel messages to a TTY.
Useful for debugging.

6 lp
→ Major number 6 is for line printers.
Historically used for parallel port printers (e.g., /dev/lp0).

Observing a Live Process

```
sleep 1000 &        # Start a process in background
PID=$!              # Capture its PID
ls /proc/$PID
```

- Shows directories and files representing the process state:
- status → Human-readable summary (threads, memory usage, state).
- maps → Virtual memory layout.
- fd/ → Open file descriptors.
- cmdline → Command-line arguments.
- environ → Environment variables.

Observing a Live Process

```
cat /proc/$PID/status | head   # Show process status
```

- Name: sleep
- Umask: 0022
- State: S (sleeping)
- Tgid: 449028
- Ngid: 0
- Pid: 449028
- PPid: 406625
- TracerPid: 0
- Uid: 923518422 923518422 923518422 923518422
- Gid: 1214800513 1214800513 1214800513 1214800513

Observing a Live Process

```
cat /proc/$PID/maps | head
```

Start	End	Perms	Offset	Dev	Inode	Path
5b7999ddc000	5b7999dde000	r--p	00000000	103:02	30147684	/usr/bin/sleep
5b7999dde000	5b7999de2000	r-xp	00002000	103:02	30147684	/usr/bin/sleep
5b7999de2000	5b7999de3000	r--p	00006000	103:02	30147684	/usr/bin/sleep
5b7999de4000	5b7999de5000	r--p	00007000	103:02	30147684	/usr/bin/sleep
5b7999de5000	5b7999de6000	rw-p	00008000	103:02	30147684	/usr/bin/sleep
5b79a2331000	5b79a2352000	rw-p	00000000	00:00	0	[heap]
7903de000000	7903de573000	r--p	00000000	103:02	30148332	/usr/lib/locale/locale-archive
7903de600000	7903de628000	r--p	00000000	103:02	30172945	/usr/lib/x86_64-linux-gnu/libc.so.6
7903de628000	7903de7bd000	r-xp	00028000	103:02	30172945	/usr/lib/x86_64-linux-gnu/libc.so.6
7903de7bd000	7903de815000	r--p	001bd000	103:02	30172945	/usr/lib/x86_64-linux-gnu/libc.so.6

Observing a Live Process

“Why do we see multiple entries for /usr/bin/sleep?”

- Expected answer: Different sections (code, data) with different permissions.

“What does [heap] mean?”

- Expected answer: Dynamic memory region for allocations.

“Why is libc.so.6 mapped?”

- Expected answer: Because sleep uses libc functions.